

UNIVERSIDAD DE ANTIOQUIA

Universidad De Antioquia

Facultad de Ingeniería

Informática II

DESAFÍO II Ude@Book

Presenta:

Luis Miguel Martinez Ocampo

Medellín, Antioquia

Octubre de 2026

Informe preliminar Desafío II

El informe presentado aquí comprende el análisis y la solución planteada para Ude@Book, teniendo en cuenta los lineamientos entregados en la guía. Se busca definir cómo se va a organizar la información, qué clases se necesitan y de qué forma se relacionan, para que al momento de implementar las funcionalidades se tenga una base clara del programa.

Ude@Book es una red social que permite manejar usuarios, amistades, solicitudes, mensajes, álbumes y fotos. En este caso el desarrollo se realizará en C++ utilizando programación orientada a objetos y memoria dinámica. La interacción se hará por consola y, cuando se pida mostrar una foto, únicamente se mostrará la ruta del archivo donde está guardada.

La información inicial se cargará desde archivos, ya que los usuarios, los álbumes y las fotos se consideran creados. Por otra parte, las operaciones se realizarán en memoria y los archivos se utilizarán para conservar los cambios. Así se evita estar consultando el almacenamiento cada vez que se necesita revisar una amistad o buscar un mensaje.

Análisis del problema

Para organizar las amistades se propone una matriz de N filas por N columnas, donde N corresponde a la cantidad de usuarios. Una posición con valor 1 indica que los dos usuarios son amigos y una posición con valor 0 indica que no lo son. Si el usuario A es amigo del usuario B, entonces B también debe ser amigo de A, por lo cual la matriz debe mantener la misma información en ambos sentidos.

Cada usuario también tendrá una lista de índices de sus amigos. Se eligió este enfoque porque la matriz permite consultar directamente una amistad, mientras que la lista permite recorrer únicamente los amigos que tiene el usuario. El costo es guardar la relación en dos estructuras, por lo cual ambas se deben actualizar juntas.

Aspectos para implementar	**Descripción**
Acceso	Cargar datos, validar credenciales y mostrar el menú.
Amistades	Enviar, aceptar o rechazar solicitudes y consultar sugerencias.
Fotos	Mostrar perfiles de amigos y recorrer álbumes permitidos.
Mensajes	Enviar, leer nuevos y buscar recibidos por fechas.
Datos y recursos	Guardar cambios y reportar memoria e iteraciones.

Esta entrega comprende el análisis y el diseño del 9 de octubre. La implementación, sus pruebas y la evidencia de funcionamiento corresponden a la siguiente etapa, con fecha del 16 de octubre.

Referencia: guía Desafío II Informática II 2026-2, versión 1.1.2, del 5 de octubre de 2026. Para la entrega final se requiere repositorio público con informe y código, commits regulares, video de 5 a 15 minutos y sustentación. En Ude@ se solicitan los enlaces al repositorio y al video; después del plazo no se debe modificar el repositorio.

Restricciones y manejo de memoria

Restricción	Consecuencia en el diseño
Sin contenedores de la STL ni structs	Crear las estructuras mediante clases propias y memoria dinámica.
Sin herencia	Relacionar las clases mediante composición y asociación.
Sin sintaxis ANSI C	Reservar y liberar con new y delete; no usar malloc ni free.
Programación modular	Separar declaraciones e implementación en .h y .cpp.
Fotos externas	Guardar y mostrar rutas absolutas Linux de archivos .png.
Sin registro ni eliminación de usuarios	Cargar las cuentas existentes y conservar sus índices.
Sin edición de álbumes y fotos	Cargar estos datos y modificar solamente los likes previstos.
Sin eliminación de amistades	Agregar relaciones al aceptar solicitudes.
Medición obligatoria	Contar ciclos directos e indirectos y memoria viva.

Acceso a los datos de la memoria

Para ubicar una amistad dentro del bloque continuo de la matriz se puede calcular la posición como:

posición = fila × N + columna

Ya que cada fila ocupa N posiciones, multiplicar la fila por N permite llegar al inicio de ese bloque y después se suma la columna. Por ejemplo, con cuatro usuarios, la relación de la fila 1 con la columna 3 queda en la posición 7. También se debe guardar la relación contraria en la fila 3 y columna 1. La diagonal se mantiene en cero porque una persona no es amiga de sí misma.

La matriz ocupará N × N × sizeof(unsigned char). Para 1.000 usuarios son 1.000.000 de bytes C++, aproximadamente 0,954 MiB en un equipo con bytes de ocho bits. A esto se deben sumar las listas y los demás objetos, por lo cual ese valor no representa toda la memoria del programa.

Para relacionar los registros se utilizarán índices y no punteros a los elementos. Se eligió esta opción porque un arreglo puede cambiar de dirección al aumentar su capacidad. El índice se conserva siempre que los registros centrales no se eliminen ni se reordenen; si se necesita otro orden, se organizará un arreglo auxiliar de índices.

Estructura de datos

La clase principal se llamará RedSocial y tendrá los registros de usuarios, mensajes y solicitudes. Cada usuario será dueño de sus álbumes y cada álbum será dueño de sus fotos. En cambio, los mensajes se guardarán una sola vez y las bandejas conservarán el índice del mismo mensaje. Así, cuando se marque como leído, el cambio se verá desde las dos bandejas sin tener que actualizar dos copias del texto.

Clase	**Información y responsabilidad**
RedSocial	Usuarios, mensajes, solicitudes, matriz, sesión y coordinación de operaciones.
Usuario	Nombre único, nacimiento, correo, clave, ciudad, país, inscripción, perfil, índices de amigos y bandejas, y álbumes.
Album y Foto	Nombre, ruta y privacidad del álbum; fecha de captura, tamaño, ruta y likes de cada foto.
Mensaje y SolicitudAmistad	Emisor y receptor por índice; texto, fecha y lectura para mensajes; estado para solicitudes.
ArregloDinamico<T> y Matriz<T>	Bloques propios de memoria, acceso y capacidad. Son las dos plantillas propuestas.
Cadena y FechaHora	Texto dinámico y comparación de fechas válidas.
Persistencia y MedidorRecursos	Lectura y guardado; conteo de iteraciones y memoria.

Manejo de las clases

Los atributos serán privados y las operaciones públicas permitirán consultar o modificar solamente lo necesario. Por ejemplo, Foto tendrá darLike y quitarLike, pero no un setter que permita dejar el contador negativo. Para consultas se utilizarán referencias constantes cuando no sea necesario modificar ni copiar el objeto.

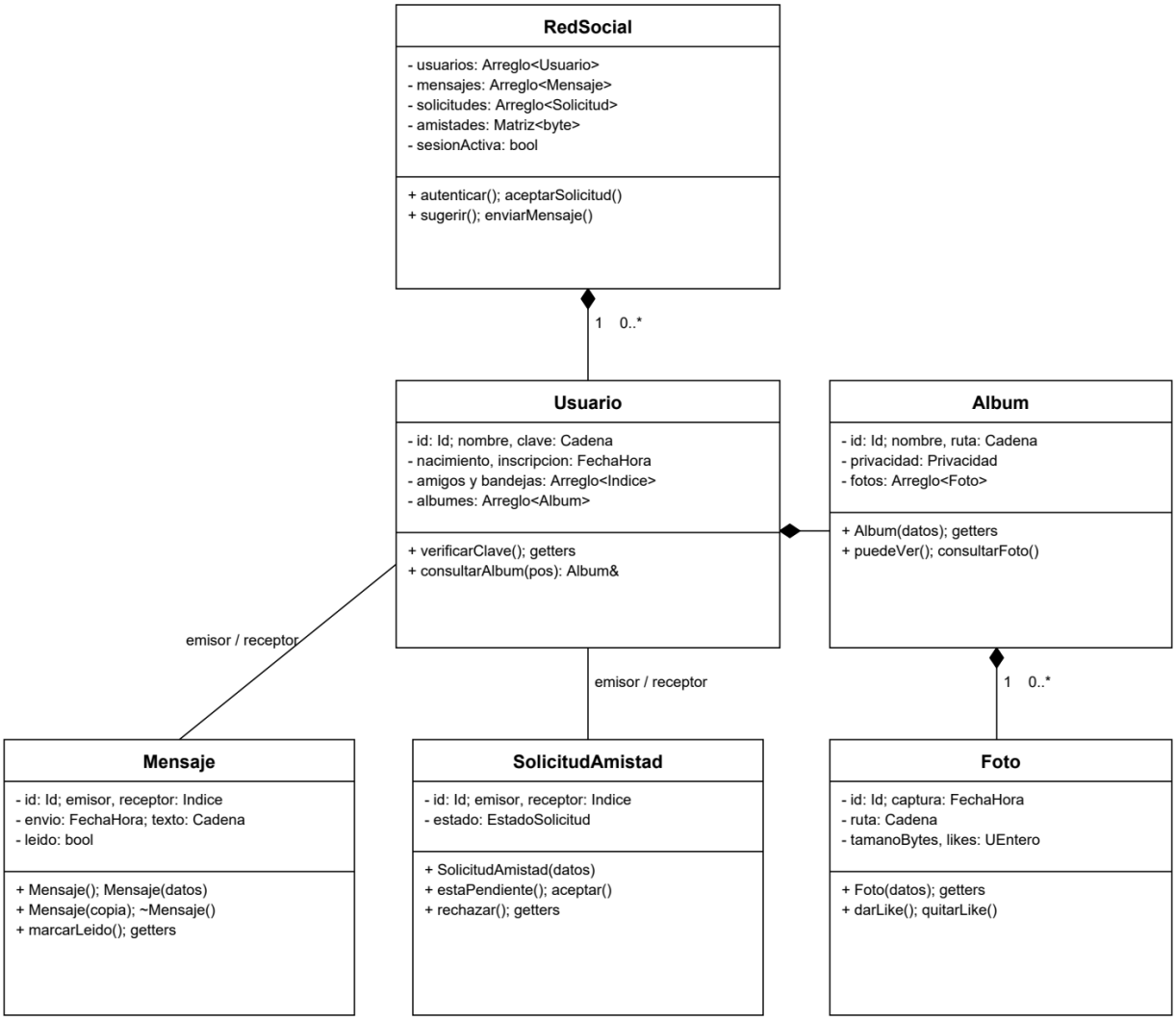
ArregloDinamico, Matriz y Cadena tendrán constructor, constructor de copia, operador de asignación y destructor con copia profunda. Esto significa que una copia tendrá su propio bloque de memoria y no compartirá por accidente el puntero del original. Las demás clases podrán apoyarse en estos miembros para copiar sus datos; RedSocial tendrá la copia deshabilitada para evitar duplicar todo el sistema.

Se sobrecargará operator[] para acceder al arreglo y los operadores < y == para comparar FechaHora. También se tendrán constructores de fecha con y sin hora, y accesos constantes y modificables con el mismo nombre. Una función amiga operator<< permitirá mostrar la fecha. No se utilizará herencia.

Las fechas sin hora, como nacimiento y captura, usarán la parte de fecha; la hora se dejará en cero. Privacidad y EstadoSolicitud serán enumeraciones con valores permitidos. Persistencia podrá ser amiga de las clases que necesite reconstruir, sin exponer modificación libre de todos los atributos.

Diagrama de clases

El diagrama presenta las clases principales. El signo menos indica atributos privados y el signo más indica operaciones públicas. Arreglo corresponde a ArregloDinamico y UEntero a unsigned long long. Los rombos llenos representan composición.



RedSocial compone 0..* usuarios, mensajes y solicitudes. Un usuario compone 0..* álbumes y cada álbum compone 0..* fotos. Cada mensaje y solicitud tiene exactamente un emisor y un receptor; un usuario puede participar en 0..* registros por cada rol. Las bandejas son asociaciones por índice y no son dueñas de los mensajes.

La amistad relaciona Usuario 0..* con Usuario 0..*, sin autorrelaciones. La línea Usuario con Album es de composición, con multiplicidad 1 a 0..*. Los atributos personales restantes de Usuario se detallan en la tabla anterior para mantener legible el dibujo. Las entidades tendrán constructor por defecto, con datos, de copia, asignación y destructor, salvo la copia de RedSocial, que estará deshabilitada.

Clases de apoyo y relaciones



RedSocial contiene una Matriz<unsigned char>, los arreglos centrales y un MedidorRecursos. Usuario contiene arreglos de índices y de Album; Album contiene un arreglo de Foto. Cadena y FechaHora se contienen por valor en las entidades que los necesitan. Las plantillas no son clases base ni implican herencia.

Persistencia depende de RedSocial por referencia. Las operaciones y las estructuras que realizan reservas utilizan el mismo MedidorRecursos por referencia; no se crea un contador separado por cada llamada. La consola, desde main, depende de RedSocial para ejecutar las opciones.

Las reservas de ArregloDinamico crecerán de manera geométrica. Se distinguirá cantidad de elementos y capacidad reservada, ya que la memoria disponible puede ser mayor que la cantidad utilizada. No se conservarán referencias a elementos después de una operación que cambie el bloque.

Tipos de datos y archivos del programa

Variable o dato	Tipo	Justificación
Textos y rutas	Cadena	Manejo propio de texto variable con memoria dinámica.
Índices, cantidad y capacidad	std::size_t	Representan posiciones y tamaños en memoria.
Identificadores	unsigned long long	Conservan la identidad del registro en los archivos.
Celda de amistad	unsigned char	Solo necesita representar los valores 0 y 1.
Lectura y sesión	bool	Son estados de dos posibilidades.
Fechas	FechaHora	Permite validar y ordenar los datos por fecha.
Privacidad y solicitud	enum class	Restringe los valores a estados definidos.
Likes, tamaño e iteraciones	unsigned long long	Son cantidades enteras no negativas.
Peso aleatorio	unsigned int	Utiliza los valores enteros 2, 5 y 7.
K de sugerencias	int	Permite rechazar negativos y validar de 1 a 20.

Se utilizará sizeof para medir los tamaños reales de los tipos en el equipo. Antes de reservar bloques o aumentar contadores se validará que la operación no desborde el tipo utilizado. Para los pesos acumulados se utilizará unsigned long long.

Propuesta de archivos del programa

Archivo	Responsabilidad	Operaciones principales
RedSocial.h y .cpp	Coordinar el sistema	Autenticar, solicitudes, sugerencias y mensajes.
Usuario, Album, Foto, Mensaje y SolicitudAmistad (.h y .cpp)	Entidades del problema	Consultar datos y aplicar cambios válidos.
Cadena y FechaHora (.h y .cpp)	Texto y fechas	Copiar, validar y comparar.
ArregloDinamico.h y Matriz.h	Plantillas propias	Reservar, copiar, liberar y acceder.
Persistencia.h y .cpp	Archivos de datos	Cargar, validar y guardar.
MedidorRecursos.h y .cpp	Recursos	Contar iteraciones y memoria.
main.cpp	Consola	Leer entradas y presentar el menú.

Las definiciones de las plantillas permanecerán en la cabecera o en un archivo incluido por ella, para que estén disponibles al compilar cada tipo. Las búsquedas y los ordenamientos esenciales se implementarán de manera propia.

Propuesta de archivos de datos

Se propone separar la información en seis archivos de texto UTF-8. Cada línea tendrá un registro y los campos estarán separados por el carácter |. La primera línea identificará la versión y la entidad, por ejemplo UDEABOOK|1|usuarios. Así se puede revisar qué contiene cada archivo y validar su formato antes de cargarlo.

Archivo	**Campos de cada registro**
usuarios.txt	id | nombre | nacimiento | correo | perfil | clave | ciudad | pais | inscripcion
amistades.txt	idUsuarioMenor | idUsuarioMayor
solicitudes.txt	id | idEmisor | idReceptor | estado
mensajes.txt	id | idEmisor | idReceptor | fechaHora | leido | texto
albumes.txt	id | idPropietario | nombre | ruta | privacidad
fotos.txt	id | idAlbum | captura | tamanoBytes | ruta | likes

Las fechas se guardarán como AAAA-MM-DD y cuando se necesite hora se usará AAAA-MM-DDTHH:MM:SS. Leído tendrá valor 0 o 1; la privacidad será PUBLICO, PRIVADO o AMIGOS; una solicitud estará PENDIENTE, ACEPTADA o RECHAZADA. Las rutas tendrán una subruta común para todos los usuarios y seguirán la notación Linux indicada en la guía.

Si un texto contiene el separador o un salto de línea, se codificará para no dividir el registro por accidente. Se cambiará % por %25, | por %7C, salto de línea por %0A y retorno por %0D. Al leer se hará una sola pasada de decodificación, sin volver a interpretar el texto ya convertido.

Carga y actualización

Primero se cargarán usuarios, después álbumes y fotos, y finalmente amistades, solicitudes y mensajes. Los identificadores del archivo se traducirán a las posiciones de memoria mediante tablas auxiliares propias. Se comprobarán nombres únicos, referencias existentes, fechas válidas, estados permitidos y valores no negativos. Una carga inconsistente se rechazará indicando el registro que presenta el problema.

Cada amistad se guardará una sola vez y al cargar se reconstruirán la matriz simétrica y las listas de ambos usuarios. Las sugerencias no se guardarán porque dependen de las amistades actuales. Los índices de las bandejas se ordenarán por fecha y por identificador cuando haya empate.

Los cambios se guardarán después de las operaciones que modifiquen datos y al cerrar normalmente. Para evitar mezclar versiones, se propone escribir los seis archivos en un conjunto nuevo y publicar un manifiesto que identifique ese conjunto cuando todos se hayan escrito correctamente. El manifiesto es un archivo auxiliar de control. Su sustitución y la recuperación ante una interrupción se verificarán al implementar; si el guardado falla, se informará y se conservará la versión anterior.

Algoritmos de amistades y selección

Ingreso y solicitudes de amistad

Para ingresar se busca el nombre en un arreglo auxiliar de índices ordenado por nombre y se verifica la clave. Se utilizará búsqueda binaria sin reordenar el registro de usuarios. Si las credenciales son correctas, se activa la sesión y se presentan el menú y las fotos de perfil de sus amigos.

Antes de enviar una solicitud se revisa que el destinatario exista, sea diferente del usuario actual, no sea su amigo y no haya una solicitud pendiente entre ambos en ninguna dirección. Para aceptarla se comprueba que corresponda al usuario conectado y siga pendiente; se reserva espacio en las dos listas, se registra la amistad en ambos sentidos y se cambia el estado. Rechazar solo cambia el estado de la solicitud.

Sugerencias de amistad

Se busca recorrer los amigos de los amigos del usuario y contar cuántas veces aparece cada candidato. Por ejemplo, si Ana es amiga de Luis y de Pedro, y ambos son amigos de Marta, entonces Marta tiene dos amigos en común con Ana. El contador se incrementa una vez por cada amigo común, por lo cual las listas no deben tener repetidos.

Primero se crea un arreglo de contadores en cero y se marcan los candidatos con solicitudes pendientes. Después se recorren los amigos directos y sus listas, excluyendo al usuario conectado, a los amigos actuales y a los pendientes. Finalmente se conservan los mejores K en un arreglo ordenado de máximo 20 posiciones, con mayor cantidad de coincidencias primero. En empate se usa el nombre en orden alfabético. Si hay menos candidatos se muestran los disponibles y se permite enviar una solicitud.

Selección de fotos de perfil

Amigos en común	Prioridad indicada	Peso utilizado
0 a 10	1	2
11 a 20	2,5	5
Más de 20	3,5	7

Se eligieron 2, 5 y 7 porque conservan la proporción sin utilizar decimales. Se suman los pesos disponibles, se genera un entero aleatorio entre 1 y esa suma y se recorre el acumulado hasta encontrar el amigo seleccionado. Después se retira del grupo de candidatos y se repite hasta completar tres. Así no se repite el mismo amigo dentro del grupo.

La actualización se permite una sola vez por sesión y excluye a los amigos mostrados inicialmente. Si no hay suficientes para completar otras tres fotos, se muestran los restantes y se informa la situación. Esta es una decisión propuesta para el caso que la guía no precisa. Los pesos son prioridades de cada extracción y no porcentajes fijos de aparición del grupo completo.

Álbumes mensajes y recursos

Recorrido de álbumes

Para visitar un perfil se verifica que sea de un amigo y se listan solamente sus álbumes públicos o con privacidad amigos. El permiso también se comprueba al abrir el álbum. Si no tiene fotos se informa y no se intenta calcular una posición. Con F fotos y posición i, la siguiente será (i + 1) % F y la anterior (i + F - 1) % F. Esto permite volver al inicio o al final sin salir del carrusel.

En cada foto se puede salir, avanzar, retroceder, dar o quitar like. Quitar solamente reduce el contador cuando es mayor que cero. No se llevará historial individual de likes, ya que en esta versión no se exige validar uno por persona.

Envío y consulta de mensajes

Se comprueba que el destinatario sea un amigo y que el texto tenga entre 1 y 512 caracteres, sin estar formado únicamente por espacios. Cadena almacenará UTF-8 y contará puntos de código para esta validación; los bytes reservados se medirán por separado. Esta interpretación de carácter deberá mantenerse en todos los componentes.

El mensaje se agrega una vez al registro central y su índice se inserta en las dos bandejas según fecha e identificador. Leer nuevos recorre los recibidos, muestra los no leídos e inmediatamente cambia su estado. Buscar por fechas valida que inicio no sea posterior al fin, localiza los límites mediante búsqueda binaria y muestra el intervalo, incluyendo completos ambos días si la entrada no tiene hora.

Conteo de iteraciones

Cada funcionalidad inicia un contador y sus métodos auxiliares utilizan el mismo por referencia. Se cuenta cada ejecución del cuerpo de un ciclo, incluyendo recorridos, comparación de textos, copias, ordenamientos y procesamiento de archivos. Si un ciclo da diez vueltas y en cada una ejecuta otro de cinco, se cuentan 10 + 50 iteraciones. Las llamadas externas, como aleatorios y entrada o salida, se reportan sin inventar sus ciclos internos.

Consumo de memoria

Se suman los objetos vivos y las reservas dinámicas, usando capacidad reservada y no solamente cantidad de elementos. Un arreglo de capacidad C ocupa C × sizeof(T), más su objeto administrador y los bloques adicionales de sus elementos. Si un atributo ya está incluido dentro de sizeof de su propietario, no se vuelve a sumar.

También se incluyen variables locales, temporales y parámetros por valor que sigan vivos al terminar la funcionalidad, antes de destruir sus variables locales. Las referencias no suman una copia del objeto referido. El registro de reservas no se reinicia al cambiar de funcionalidad, porque la memoria sigue ocupada. El valor corresponde a los datos y objetos contabilizados; no se presentará como toda la RAM del proceso, cuyo costo interno no se conoce de forma portátil.

Consideraciones para la implementación

Eficiencia de las operaciones

Operación	Costo previsto y consideración
Comprobar amistad	O(1) mediante la matriz, después de localizar los usuarios.
Buscar usuario	O(log N) comparaciones de nombres; cada comparación depende de la longitud del texto.
Sugerencias	O(N + P + suma de grados de los amigos + N × K), sin incluir la longitud de desempates por nombre. P son solicitudes examinadas.
Seleccionar perfiles	Contar coincidencias cuesta O(suma de grados de los amigos). Para tres sorteos, el recorrido ponderado cuesta O(d), con d amigos.
Consultar mensajes por fechas	O(log R + Q), con R recibidos y Q resultados.
Insertar en una bandeja	Hasta O(R) por desplazamiento de índices.
Mover el carrusel	O(1) para calcular la posición; mostrar la ruta depende de su longitud.

Decisiones frente a casos no definidos

Se adopta orden cronológico ascendente para mensajes, desempate alfabético en sugerencias y exclusión de solicitudes pendientes de los candidatos. La actualización de fotos se entiende una vez por sesión. Se mostrará una cantidad menor cuando falten candidatos. Si distintas cuentas comparten una misma ruta de perfil, se evitará repetir esa ruta dentro del grupo; esto puede reducir la cantidad mostrada. Rechazar solicitudes se incluye porque aparece en la descripción del usuario, aunque no se desarrolla en el menú esencial.

Alternativas y dificultades previstas

Usar solamente la matriz simplifica mantener los datos, pero obliga a recorrer filas completas para enumerar amigos. Utilizar solo listas ahorra la matriz, pero hace más costosa la consulta directa de amistad. En este caso se propone conservar ambas para el escenario solicitado, teniendo presente que el consumo de la matriz crece con N al cuadrado.

Los puntos que requieren mayor cuidado serán mantener las relaciones sincronizadas, copiar y liberar memoria sin duplicar propietarios y contar los recursos dentro de los métodos auxiliares. Son dificultades previstas del diseño. Los problemas encontrados al programar y los cambios reales de la solución se registrarán durante la implementación, sin presentarlos aquí como hechos ya ocurridos.

Validación prevista

Se probarán usuarios sin amigos, menos de seis amigos para los grupos de fotos, pesos en 10, 11, 20 y 21 coincidencias, solicitudes cruzadas, álbumes vacíos y privados, mensajes vacíos y de 512 y 513 caracteres, fechas límite y persistencia del estado leído. La demostración final deberá usar mínimo 1.000 usuarios, consultar 12 sugerencias, enviar solicitud al de mayor coincidencia, buscar mensajes en seis semanas y recorrer dos álbumes de cinco fotos, mostrando recursos. Estos son casos previstos y no pruebas realizadas.